Les Tableaux Unidimensionnels et le Passage par Adresse en Langage C

Thème principal : Allocation mémoire contiguë, calcul d'adresse O(1), passage par adresse et mécanisme d'Array Decay en C

Synthèse exécutive : Ce cours pose les fondements algorithmiques et système du stockage de séquences de données homogènes en langage C. Il explicite la disposition contiguë des tableaux unidimensionnels en mémoire vive (stack), garantissant un accès direct à tout élément en complexité temporelle constante O(1) grâce à une formule d'arithmétique système d'offset. Il décortique le mécanisme d'Array Decay : lors de la transmission d'un tableau à une fonction, le nom du tableau s'évalue automatiquement comme un pointeur vers son premier élément. Cela implique deux règles fondamentales : d'une part, les modifications effectuées dans la fonction s'appliquent directement sur le tableau d'origine (passage par adresse) ; d'autre part, la dimensionnalité du tableau est perdue dans le scope de la fonction, rendant le calcul dynamique de taille par sizeof impossible et exigeant le passage explicite de la taille en paramètre.

1. Représentation mémoire, contiguïté et calcul d'adresse en O(1)

Lorsqu'un tableau statique est déclaré en langage C (par exemple `int tab[5];`), le compilateur ordonne la réservation d'un bloc continu en mémoire vive (sur la pile / stack). Pour 5 entiers codés sur 4 octets chacun, le système alloue un espace global ininterrompu de 20 octets consécutifs (`5 * sizeof(int)`). L'accès à un élément via la syntaxe crochet `tab[i]` ne réalise aucun parcours séquentiel des éléments précédents. Le système calcule instantanément l'adresse mémoire exacte de la case ciblée en appliquant la formule d'arithmétique système suivante : `Adresse(tab[i]) = Adresse_Base + (i * sizeof(type))` Puisque le premier élément ne présente aucun décalage par rapport à l'adresse de base du bloc (`Adresse_Base + (0 * sizeof(type))`), son indice est impérativement 0. Ainsi, pour un tableau de taille N, les indices valides s'étendent exclusivement de `0` à `N - 1`. Tenter d'accéder à l'indice `N` (ex: `tab[5]` pour un tableau déclaré avec `int tab[5]`) constitue un débordement de mémoire (Buffer Overflow). En effet, le langage C n'effectue aucun contrôle de bornes à l'exécution pour privilégier la vitesse. Accéder à un indice hors limites revient à lire ou écraser la mémoire voisine (autres variables ou pointeurs), ce qui entraîne des résultats incohérents ou un plantage immédiat (Segmentation Fault).

- Tableau unidimensionnel : Structure de données statique et homogène stockant un nombre fixe d'éléments de même type dans la mémoire.
- Contiguïté mémoire : Réservation d'une plage continue d'adresses mémoire ininterrompue de N * sizeof(type) octets sur la pile.
- Indexation de 0 à N-1 : L'indice i représente le décalage (offset) relatif à l'adresse de base de la première case.
- Complexité constante O(1) : Accès immédiat à n'importe quelle case par calcul direct d'adresse système sans parcours séquentiel.
- Buffer Overflow : Risque d'accès hors limites (ex: tab[N] pour un tableau de taille N) provoquant un comportement indéterminé ou une Erreur de Segmentation.

Cas concret enseignant : *Le professeur a illustré la contiguïté mémoire à l'aide d'une analogie concrète : un meuble de rangement vertical comportant des tiroirs rigoureusement identiques et collés les uns aux autres, numérotés de 0 à N-1. Si l'on connaît l'emplacement exact du sol (l'adresse de base de la structure) et la hauteur précise d'un tiroir (sizeof(type)), on calcule instantanément la position du tiroir i en appliquant Sol + (i * Hauteur), sans avoir à ouvrir ou mesurer tous les tiroirs du dessous un par un.*

2. Passage de tableaux aux fonctions, pointeurs et mécanisme d'Array Decay

En langage C, la transmission d'un tableau par valeur (copie intégrale de l'ensemble des éléments) est proscrite pour éviter de dégrader les performances du processeur et de surcharger la pile mémoire. Dès lors que l'identifiant

d'un tableau est passé en paramètre à une fonction (ex: `calculer(tab)`), le langage applique un mécanisme fondamental appelé 'Array Decay' (dégradation du tableau). Le nom du tableau s'évalue automatiquement sous la forme d'un pointeur vers son tout premier élément (`&tab;[0]`). Deux règles techniques majeures en découleront dans vos programmes : 1. Passage par adresse effectif : Le paramètre formel dans la fonction réceptrice (déclaré `int tab[]` ou `int *tab`) pointe directement vers les cases mémoire d'origine. Toute modification apportée aux éléments du tableau à l'intérieur de la fonction altère directement le tableau de la fonction appelante. 2. Perte d'information de taille : À l'intérieur de la fonction réceptrice, le type d'origine 'tableau de N éléments' s'est dégradé en un type pointeur (`int*`). En conséquence, l'utilisation de l'opérateur `sizeof(tab)` au sein de la fonction ne renvoie pas la taille du tableau complet, mais simplement la taille physique du pointeur (4 octets sur une architecture 32-bit, 8 octets sur une architecture 64-bit). C'est pourquoi une fonction manipulant un tableau C doit obligatoirement recevoir sa taille explicite en paramètre (ex: `int taille`).

- • Array Decay : Conversion implicite et automatique du nom d'un tableau en un pointeur de type `type*` vers son premier élément (`&tab;[0]`) lorsqu'il est transmis à une fonction.
- • Passage par adresse : Transmission de l'emplacement mémoire original évitant toute copie lourde de structures de données volumineuses.
- • Arithmétique des pointeurs : Équivalence stricte en C entre l'écriture indexée `tab[i]` et l'accès par pointeur `*(tab + i)`.
- • Perte de dimensionnalité : Impossibilité pour une fonction de calculer dynamiquement la taille d'un tableau reçu via l'opérateur sizeof.
- • Obligation de taille : Nécessité stricte d'inclure la taille effective du tableau sous forme de paramètre complémentaire dans la signature de la fonction.

Cas concret enseignant : *Pour faire assimiler la différence fondamentale entre le passage par valeur et le passage par adresse causé par l'Array Decay, l'enseignant a expliqué que passer un tableau à une fonction ne revient pas à fabriquer une réplique exacte de sa maison dans le jardin de son ami (passage par valeur), mais à lui fournir les coordonnées GPS précises de sa propre maison (passage par adresse). L'ami se déplace en utilisant le pointeur GPS et repeint directement les murs du bâtiment d'origine.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante-t-il ou affiche-t-il des valeurs absurdes lorsque j'essaie d'accéder à tab[5] alors que mon tableau a été déclaré avec une taille de 5 (int tab[5];) ?
Réponse du professeur : C'est un piège d'indexation classique qui provoque un débordement de mémoire (Buffer Overflow). La déclaration `int tab[5]` alloue 5 cases en mémoire, mais le premier élément étant positionné à l'index 0, les 5 cases sont indexées de 0 à 4 (0, 1, 2, 3, 4). L'indice 5 désigne une 6ème case située hors des limites allouées. Comme le C ne bloque pas cet accès à l'exécution, le processeur lit ou écrit à l'adresse de base + (5 * 4 octets), écrasant d'autres données en mémoire, ce qui provoque une Erreur de Segmentation (Segmentation Fault) ou l'affichage de valeurs indéterminées.

Question étudiant : Est-ce qu'on peut calculer la taille d'un tableau à l'intérieur d'une fonction avec l'expression sizeof(tab) / sizeof(tab[0]) ?
Réponse du professeur : Absolument pas à l'intérieur d'une fonction réceptrice, et c'est un motif d'échec fréquent en partiel. L'expression `sizeof(tab) / sizeof(tab[0])` ne fonctionne correctement que dans le scope où le tableau a été déclaré localement. Dès que le tableau est transmis à une fonction, le mécanisme d'Array Decay le dégrade en un simple pointeur `int*`. Dans la fonction, `sizeof(tab)` renverra la taille du pointeur (8 octets) et non celle du tableau. Vous obtiendrez par exemple `8 / 4 = 2`, peu importe si votre tableau comptait 10 ou 1000 éléments. C'est pour cette raison qu'il faut obligatoirement passer la taille en paramètre d'entrée de la fonction.

■ Exercice & Démonstration d'Application

Énoncé : Écrire une fonction C nommée `remplir_et_calculer_moyenne` qui prend en paramètre un tableau d'entiers et sa taille effective. La fonction doit demander à l'utilisateur de saisir au clavier chaque note pour remplir le tableau, puis calculer et retourner la moyenne arithmétique exacte des éléments sous la forme d'un nombre à virgule (`double`). La fonction doit être sécurisée contre les risques de division par zéro en cas de taille nulle ou

négative.

Résolution pas à pas :
1. Signature de la fonction : Déclarer `double remplir_et_calculer_moyenne(int tab[], int taille)` (ou `int *tab`). 2. Validation de sécurité : Tester si `taille <= 0`. Si c'est le cas, retourner `0.0` pour empêcher toute division par zéro au niveau système. 3. Initialisation de l'accumulateur : Déclarer une variable `double somme = 0.0;` pour accumuler les valeurs saisies tout en imposant l'arithmétique en valeur décimale (`double`). 4. Boucle de parcours et de saisie : Implémenter une boucle `for` avec un indice `i` allant de `0` à `taille - 1`. Pour chaque passe, demander la saisie via `scanf("%d", &tab;[i])` (l'adresse de la case `tab[i]`) et cumuler directement la note dans `somme`. 5. Calcul et retour du résultat : Diviser la `somme` par la `taille` (conversion implicite en `double`) et retourner le quotient.

Résultat : #include double remplir_et_calculer_moyenne(int tab[], int taille) { // 1. Sécurisation contre la division par zéro if (taille <= 0) { return 0.0; } double somme = 0.0; // 2. Parcours, saisie et accumulation for (int i = 0; i < taille; i++) { printf("Saisir la note n°%d : ", i + 1); scanf("%d", &tab;[i]); // Saisie à l'adresse de la case i somme += tab[i]; } // 3. Calcul de la moyenne arithmétique exacte return somme / taille; } // Exemple de test d'exécution dans le main() : // int notes[3]; // double moyenne = remplir_et_calculer_moyenne(notes, 3); // Pour des saisies de 12, 14, 16 -> La fonction retourne 14.000000

■ Auto-Évaluation Flash d'Examen

Q : Quel est l'indice du dernier élément accessible d'un tableau déclaré par `int t[10];` et que se passe-t-il si l'on tente d'écrire dans `t[10]` ?

R : L'indice du dernier élément est 9 (car l'indexation va de 0 à N-1, soit de 0 à 9). Tenter d'écrire dans `t[10]` constitue un débordement de mémoire (Buffer Overflow) car cela pointe vers la 11ème case non allouée, provoquant un comportement indéterminé ou une Erreur de Segmentation.

Q : Que contient l'identifiant d'un tableau `tab` lorsqu'il est utilisé seul sans crochets dans une expression C ou lors d'un passage en argument à une fonction ?

R : En raison du mécanisme d'Array Decay, l'identifiant `tab` employé seul contient l'adresse mémoire du premier élément du tableau, équivalent strictement à `&tab;[0]`. C'est un pointeur vers le début du bloc alloué.

Q : Pourquoi la signature d'une fonction manipulant un tableau C doit-elle obligatoirement inclure sa taille en paramètre séparé ?

R : Lors du passage d'un tableau à une fonction, l'Array Decay convertit le tableau en un simple pointeur `type*`. La fonction réceptrice perd la dimensionnalité du tableau et ne possède aucun moyen natif de déterminer dynamiquement son nombre d'éléments (`sizeof` renverra uniquement la taille du pointeur, soit 4 ou 8 octets).